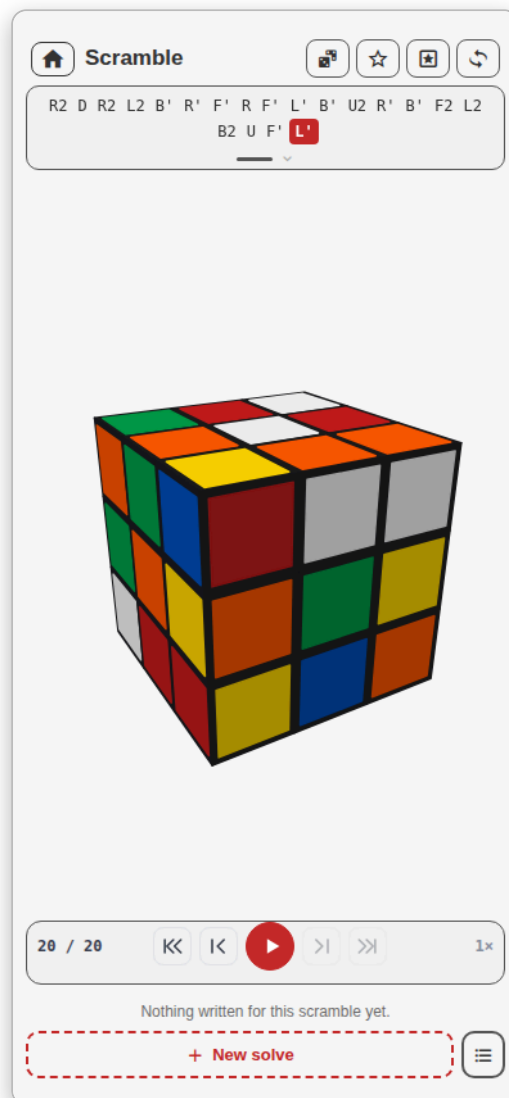


Cube Scramble

Current functionality, controls and states · redesign inventory

Current purpose: a companion for the physical 3×3 cube: generate and inspect scrambles, record/replay solves, mark method stages, curate algorithms and methods, and derive practice progress.



Scramble home.

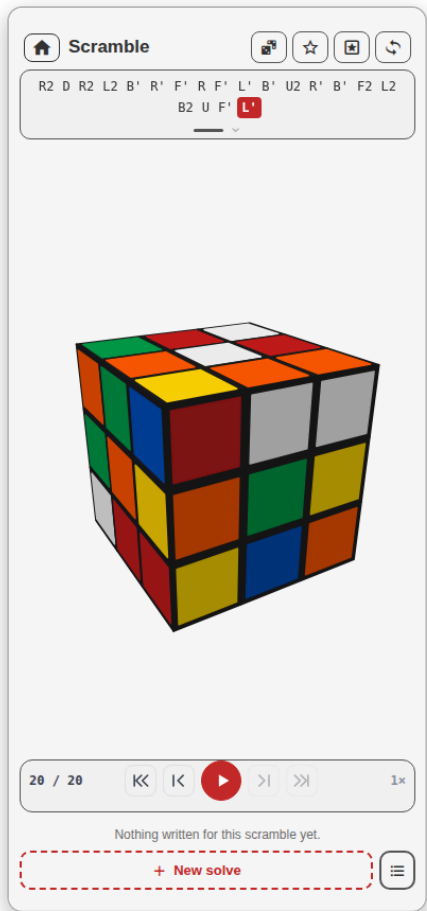
1. Current information architecture

Scramble home
↳ Solve recorder
↳ Algorithms library → Algorithm workbench / entry
→ Methods catalogue / builder
→ Journey

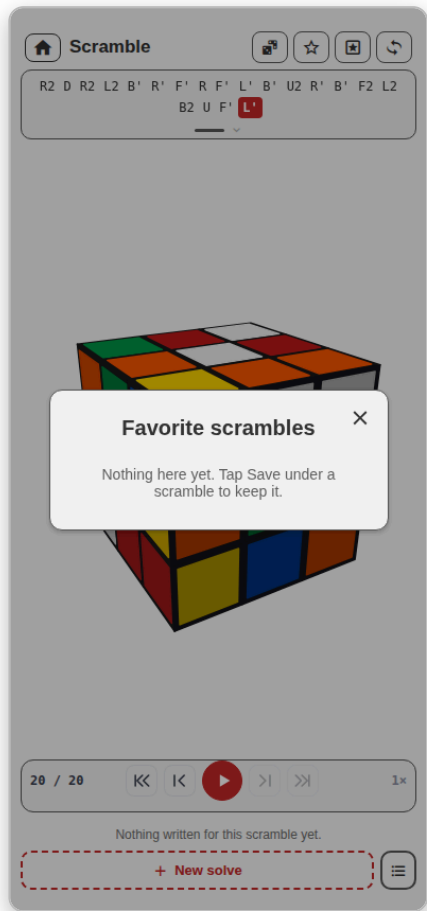
Surface	Job	Owns/edits
Scramble	Generate, animate, inspect, save and revisit scrambles; list attempts.	Current scramble, favorites, solve cards.
Solve	Record physical turns, inspect playback, set orientation, lock method boundaries and tag algorithm runs.	One open solve.
Algorithms	Search/filter the personal algorithm library and reach advanced management.	Library projection; entry/workbench edit records.
Methods	Browse presets, duplicate into user variants, rename/reorder stages and manage assignments.	User method catalogue.
Journey	Show derived beginner-to-advanced practice progress.	Nothing stored; reads solves/checks.

Important domain distinction: the rendered cube is not the physical input itself. On native, finger turns can write moves in authoring contexts; on web, cube gestures orbit only.

2. Scramble home



Scramble, cube, player and attempt list.

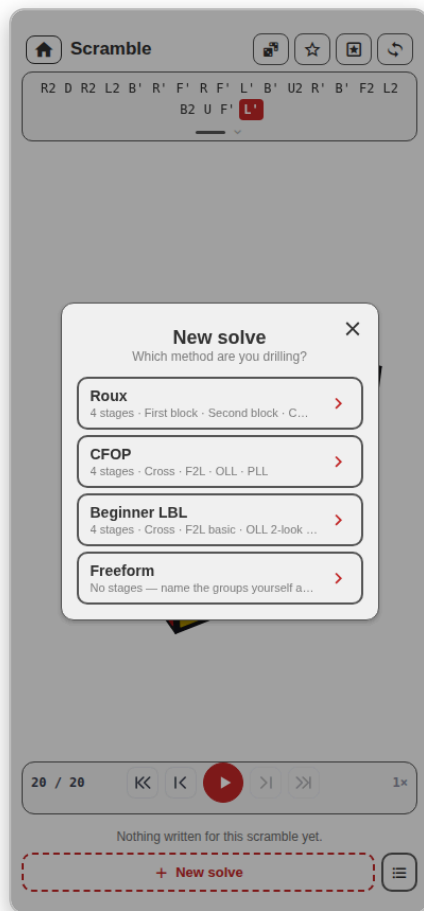


Favorite-scramble modal.

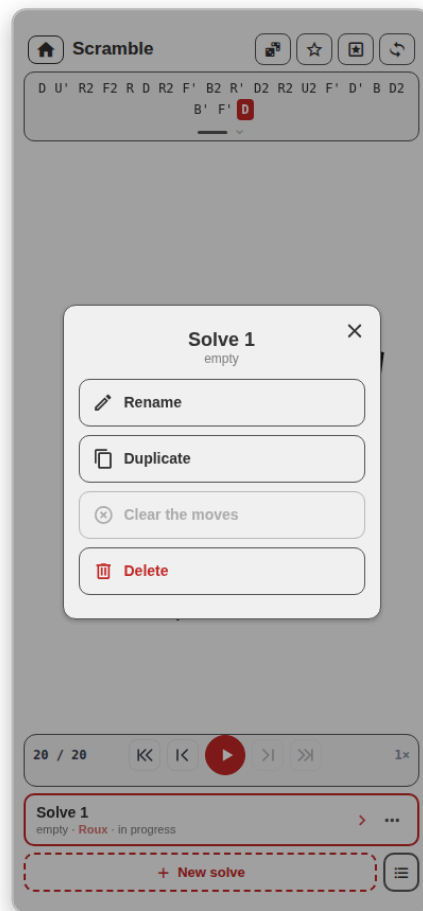
Control	What it does
Back to games	Returns to hub.
New scramble	Generates a fresh valid 20-move scramble and resets the player to its end.
Save / unsave	Toggles the current scramble in Favorites.
Favorites	Opens saved scrambles; load or delete an item.
Other side / orbit	Moves the view to the opposite three faces; dragging orbits the cube.
Notation token	Seeks playback to that move.
Show whole thing	Expands/collapses long notation presentation.
Scrubber	Seeks continuously through the move sequence.
Start / previous / play / next / end	Transport controls for scramble animation.
Turn speed	Cycles playback rate.

New solve	Opens method choice and creates an attempt.
Compare	Appears with enough attempts; compares stages/counts across solves.
Algorithms	Pushes the personal library.
Solve card	Opens that attempt; highlights current/in-progress state.
Solve card menu	Rename, duplicate or delete attempt.

3. Starting and managing solves



Method picker.



Attempt menu.

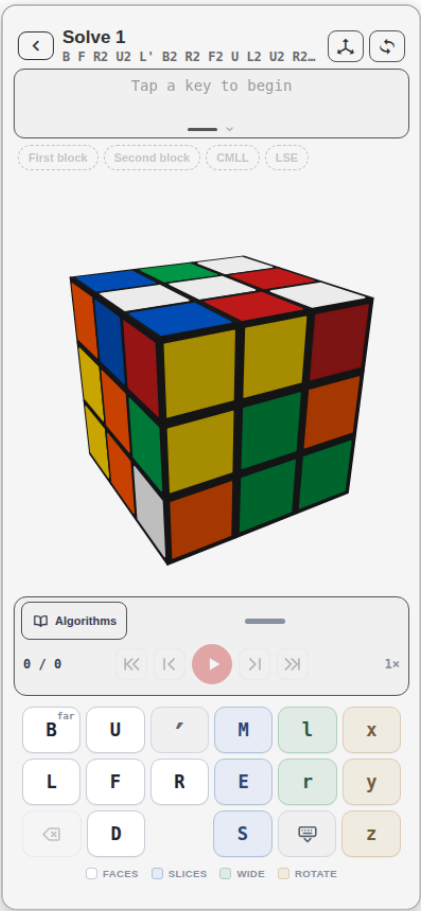
Method picker

- Preset choices: Beginner LBL, CFOP and Roux, each with its stage summary.
- User methods marked “Use for new solves” join the list.
- Freeform creates a solve with no prescribed rail and allows authored phase labels.
- Selection creates the solve immediately; there is no separate confirmation screen.

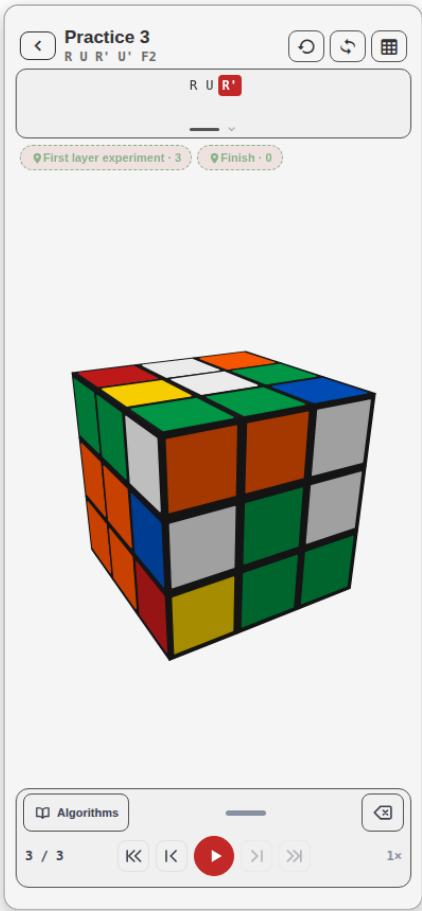
Solve-card information

- Name, method, saved/edited recency and completion/in-progress state.
- Stage summary derived from markers rather than stored ranges.
- Tagged algorithm-run names can appear in solve detail/playback.

4. Solve recorder and playback



Solve with move pad.

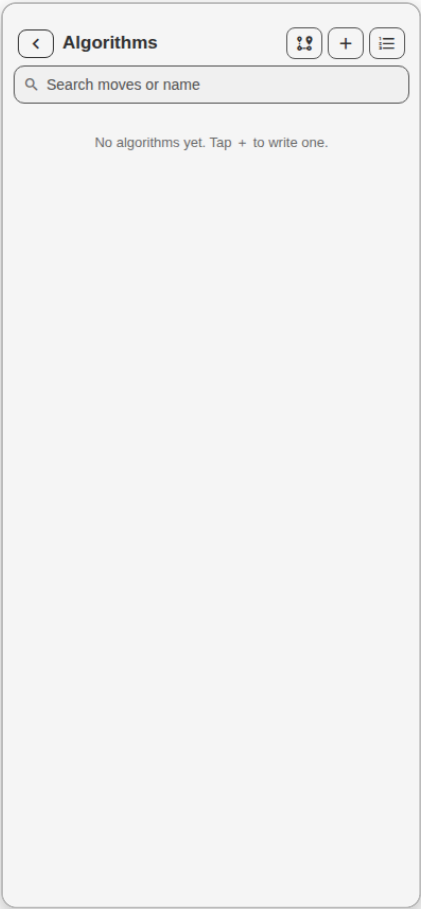


Collapsed move-pad state.

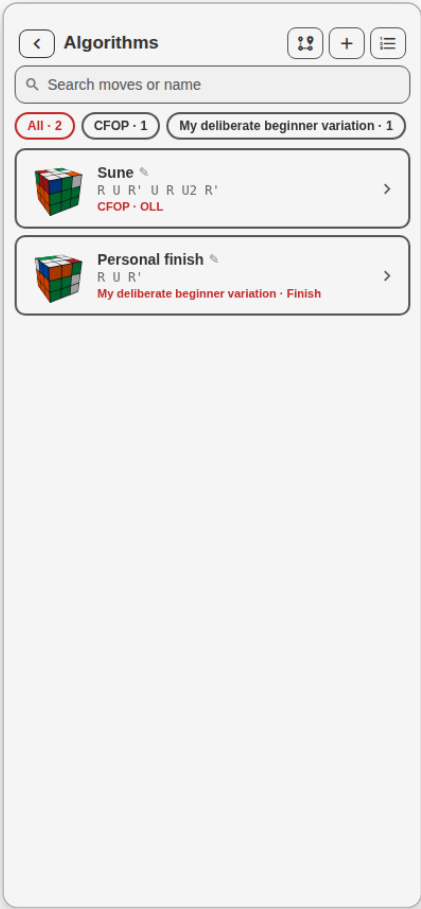
Control / gesture	What it does
Orientation / hold	Before moves, choose a whole-cube starting orientation. "Inspecting" (^null"), confirmed no rotation ("") and notation are distinct states.
Finger turn	On native, turning a sticker writes the corresponding move after the gesture settles.
Move pad	Writes face, slice, wide and rotation moves; Prime modifies the next/base move.
Type an algorithm	Pastes/validates notation as a secondary input path.
Undo last move	Removes the final authored move.
Pad handle	Shows/hides the manual drawer without changing behavior of the cube.
Transport/scrubber	Seeks and animates authored solve notation.
Stage rail	Displays method stages, current stage and boundary exit-check status.
Lock/place boundary	Places or moves a stage marker at the current scrubber cursor.

Algorithms	Opens solve-side sheet to apply an entry or save/tag a selected run.
Run marker	Shows named algorithm boundary; toggles folded/expanded move display.

5. Algorithms library



Empty library.



Seeded library with cases/assignments.

Control	What it does
Back	Returns to scramble or prior method-stage view.
Journey	Pushes derived practice track.
+	Opens cube-first workbench; disabled when the capped library is full.
Methods	Opens catalogue/builder.
Search	Matches algorithm name or notation.
Filter chips	All, Unassigned and only method filters that lead to entries; counts are live.
Entry card	Shows three-face starting case, name, moves and stage assignments; opens entry.
Stage assignment mode	From a method stage, toggles between linked-only list and whole-library assignment.

Library semantics: authored entries are durable personal work. The cap refuses new entries rather than evicting old ones.

6. Algorithm entry

< Sune

NAME

Sune

MOVES

Edit on the cube

Paste an algorithm · R U R' U R U2 R'

7 moves

CASE

The starting case you set on the cube.

USED FOR

Roux

First blockSecond blockCMLLLSE

CFOP

CrossF2L

- OLL
- PLL

Beginner LBL

CrossF2L basicOLL 2-lookPLL 2-look

My deliberate beginner variation

First layer experimentFinish

NOTES

Orient the last layer.

Delete

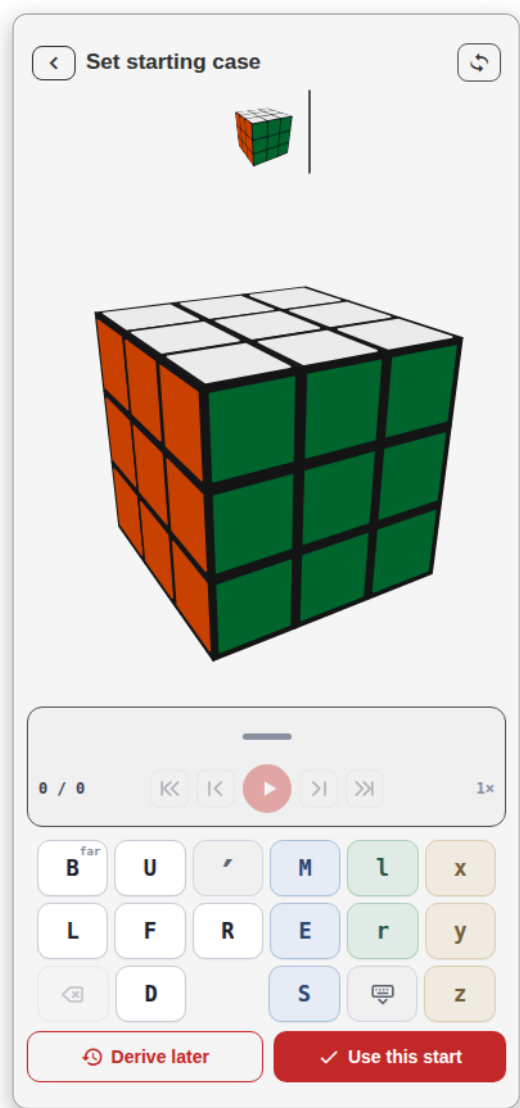
A saved algorithm entry.

Field/control	Function
Name	Editable unique display name; duplicates receive a suffix.
Three-face preview	Shows authored setup/start, or inverse-derived start for older/pasted entries.
Moves	Validated algorithm notation; required.
Paste/edit notation	Opens parameterized text modal as a secondary input path.
Assignments	Links the entry to any method/stage pair; reciprocal with method-stage library views.
Notes	Free text saved on every edit.
Delete	Removes personal entry after confirmation.

Case behavior

- A compact U-face pattern exists for description/correction data.
- The visible preview uses three faces because a flat U tile cannot distinguish PLL side stickers.
- Sticker colours depict the physical cube and intentionally do not follow app themes.

7. Cube-first algorithm workbench



Workbench at authored-start step.

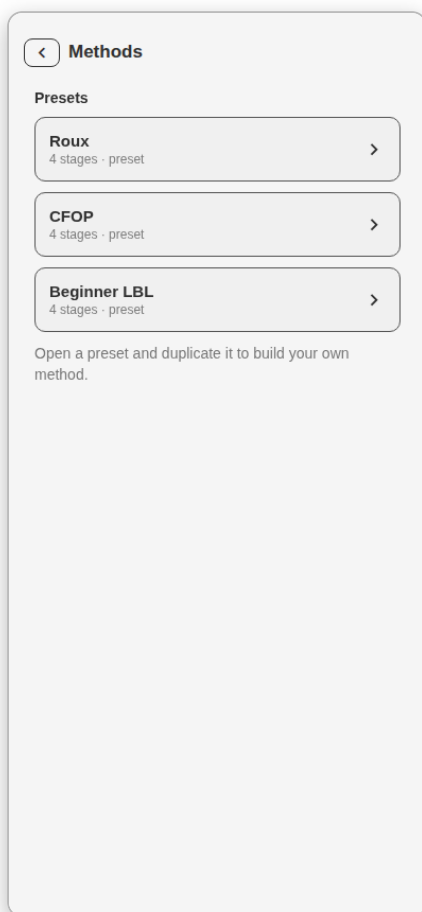
Flow

1. Start from solved and turn the cube to the position where the algorithm begins.
2. Lock the authored starting state, or explicitly derive it later/use the inverse.
3. Write the algorithm with native finger turns, move pad, or paste notation.
4. Scrub/play the result and tap the shared 3D preview to return to the start.
5. Save name and stage assignments through the shared save sheet.

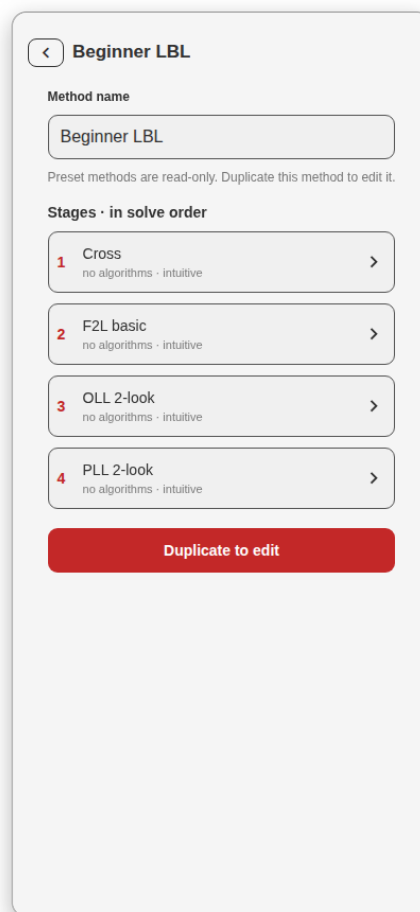
Capability	Why it exists
Authored setup	An inverse is not always the actual position from which the player practices.
Inverse fallback	Older/pasted algorithms still get a deterministic starting case.
Large cube + compact preview	Separates active authoring from the stable reference state.

Shared transport/track/pad	Keeps solve and workbench notation behavior aligned.
----------------------------	--

8. Methods catalogue and builder



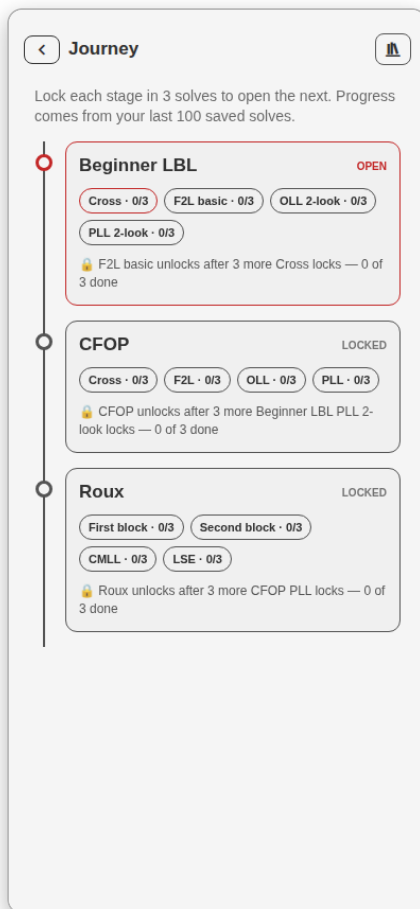
Preset/user catalogue.



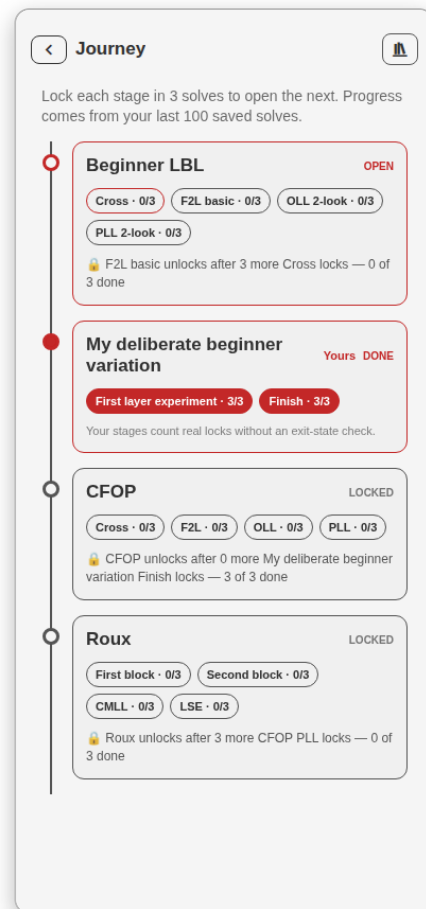
Read-only preset detail.

Control	What it does
Method row	Opens stages, linked-algorithm counts and editing surface.
Stage row	Opens exact method/stage library view.
Duplicate to edit	Creates a user variant with its own id and `from` ancestry.
Method name	Editable for user variants; uniqueness enforced.
Stage name	Renames the stage and matching solve markers/algorithm assignments in one operation.
Up/down	Reorders user stages.
Add/remove stage	Changes user stage vocabulary within caps; duplicates refused.
Use for new solves	Shows/hides user variant in the solve method picker without deleting it.
Duplicate method	Creates another independent variant.
Delete method	Refused while a saved solve still references it.

9. Journey



Fresh journey.



User variant and derived progress.

- Preset order is Beginner LBL → CFOP → Roux.
- User variants sit immediately after their source; “Yours” identifies them.
- Stages and methods are **Done**, **Open** or **Locked**.
- Three demonstrations complete a stage; gate copy names prerequisite and remaining locks.
- A shipped preset demonstration requires its locked boundary to pass a stage-specific cube-state check.
- A user stage has no pretend predicate; a genuine lock counts and the card says it is unverified.
- At most one matching marker per stage per solve counts.
- Progress is derived from the last 100 live solves. Delete/edit the evidence and progress rolls back.

No journey progress record exists. The screen replays each live solve once, consumes its stage-check matrix and projects cards.

10. Compare, tags and secondary states

Feature/state	Current behavior
Compare	Available after multiple attempts for one scramble; aligns methods/stages and exact move counts in a modal table.
Apply saved algorithm	From solve Algorithms sheet, appends only the saved moves at the live end and visibly plays them.
Save/tag run	Select token-bounded moves; rejects cross-stage selection; stores a named end-exclusive annotation without rewriting solve notation.
Fold run	Named boundary can collapse underlying moves into a chip and expand them again.
Exit-state verification	Preset stage rail distinguishes a passing boundary from a failed one; user stage is unverified, not failed.
Resume	Cube keeps its nested navigator in memory on background/resume. Authored work and view persist; scrub position and speed rewind.
Corrupt/older save	Collections sanitize independently by shape; absent newer fields become empty rather than requiring migration.

Storage vocabulary

`{ scramble, favorites, solves, algorithms, methods, workspace }`. A solve includes scramble, name, method, orientation, notation, phase markers, tagged runs and timestamps. Phase ranges and journey progress are derived.

11. Product/design discussion prompts

Primary job: Is Cube Scramble chiefly a scramble viewer, a solve recorder, an algorithm notebook, a method coach, or a journey? Today it is all five.

Novice versus expert: Beginner LBL and locked progression coexist with notation-heavy authoring, slice/wide moves and reciprocal stage assignments. Are these one audience?

Navigation: The solve list is the door to Algorithms; Algorithms is the door to Methods and Journey. Should these be peer destinations, progressive disclosure, or separate modes?

Physical/digital relationship: Which actions should happen on the real cube, the rendered cube, the pad or text? Web and native currently differ.

Evidence/progress: Is “three verified locked boundaries” the desired measure of learning, and should deleting a solve really erase progress?

Screen density: Cube size was protected by strict point budgets, producing drawers, sheets and icon doors. A redesign can reconsider the whole hierarchy rather than shuffle those controls.

Personal library: Are algorithms and user methods core user-generated content worthy of a dedicated home, or advanced tooling behind practice?

Do not lose in redesign: valid move notation, physical cube colour fidelity, orientation's three states, one writer per collection, phase markers as span starts, solve playback, edit rollback, user/preset predicate distinction, native back/edge gestures, and authored data persistence.

12. Designer handoff checklist

Any proposed structure should explicitly place or retire each capability:

- Generate scramble
- Save/load favorites
- Orbit/other side
- Scramble transport
- Create/select/rename/duplicate/delete solve
- Choose preset/user/freeform method
- Set orientation
- Write finger/pad/pasted moves
- Undo and playback
- Place/move stage boundaries
- See boundary checks
- Compare attempts
- Apply/tag/fold algorithms
- Search/filter library
- Author/edit/delete algorithm
- Author starting case
- Assign stages reciprocally
- Browse/duplicate/edit methods
- Rename/reorder/add/remove stages
- Toggle method availability
- View journey/gates/progress
- Resume and recover stored work

Screenshots are web renders at 393 × 852. Native-only finger turns, haptics, keyboard avoidance, iOS edge-back and Android hardware-back are functionality but cannot be represented faithfully in these stills.